

**FACULTY OF NATURAL SCIENCE
DEPARTMENT OF COMPUTER SCIENCE**

CSE 2101: Software Engineering

**Project Plan: Student Records Management System (SRMS) Mobile
Application**



Client: Software Services & Educational Technology Applications
(SSETA)

Date: November 7, 2025

Group 5

Kaylee Carter (1051799)

Khaleel Khan (1054193)

Ravindra Persaud (1052507)

Vimla Persaud (1052339)

Tulsidai Singh (1052682)

Table of Contents

1.0	Introduction	3
2.0	Project Organization	5
2.1	Team Structure	5
2.2	Communication and Reporting	6
2.3	Decision-Making and Accountability	6
3.0	Risk Analysis	7
4.0	Hardware and Software Resource Requirements	14
4.1	Hardware Requirements.....	14
4.2	Software Requirements	15
5.0	Work Breakdown	17
6.0	Project Schedule.....	22
	Collaborative Summary	23

1.0 Introduction

The Student Records Management System (SRMS) Mobile Application Project is designed to revolutionize the way students at the University of Guyana manage their academic responsibilities by creating a modern, efficient, and user-centric mobile platform. Mr. Tyreke Wilson will oversee this project to create a complete mobile application which unites all university operations into one secure platform for students to handle their academic tasks through their smartphones.

Students mainly access different services through the SRMS web portal at present. The system faces major problems when users try to access it through mobile devices because its design was made for laptop screens. Students face two main problems when using the system because they struggle with interface navigation and experience connectivity problems and performance issues from high user traffic. The mobile application serves as a dedicated solution to improve SRMS functionality through an easy-to-use interface which connects directly to the university's SRMS database system.

The mobile application will start by offering basic services to students during its first development stage. The system will enable users to perform multiple functions including admission application submission, document uploading, application status tracking, course selection management and immediate academic update notifications. The system provides direct access to student information which leads to better student satisfaction and higher student involvement.

The system will get upcoming updates which will let university staff members access limited features to manage grades and verify student records. The complete system enables students and staff members to move through academic procedures with ease which creates an improved learning space.

The development of this mobile application will adhere to a plan-driven (Waterfall) model, as outlined by Sommerville (2016). This structured methodology allows for thorough requirements gathering followed by systematic design, implementation, testing, and deployment phases. Each stage of development will require meticulous documentation and formal approval before progressing to the next phase, which is instrumental in maintaining rigorous quality control throughout the project lifecycle. This careful planning is crucial to ensure that the final product meets the highest standards of functionality and user experience, ultimately contributing to the academic success and satisfaction of the University of Guyana's student body.

Key constraints that affect the management of this project include:

- Time: The estimated completion period is 12 weeks. Each phase must be completed within its assigned duration to keep the project on track.

- Budget: Development will rely primarily on open-source software and university-provided infrastructure, minimizing costs but limiting access to some premium testing tools.
- Technical constraints: Integration with the existing SRMS database must be done securely and efficiently to maintain data consistency and protect sensitive student information.
- Scope limitations: The first version of the application will focus only on student functionality, with staff features considered for later development.

2.0 Project Organization

2.1 Team Structure

The SRMS mobile application project follows a functional team structure designed to promote collaboration, accountability, and efficient communication across all development phases. The structure ensures each team member contributes expertise in their respective areas while maintaining alignment with the project's objectives, timeline, and quality standards.

Role	Name	Responsibilities
Project Manager & Lead Systems Analyst	Ravindra Peraud	• Serves as the primary point of contact for the client, Mr. Tyreke Wilson, and other university stakeholders. • Oversees the project timeline, budget, and scope, ensuring milestones from the Gantt chart are met. • Leads the risk management process, actively monitoring the high-priority risks identified. • Translates client needs and user stories into detailed functional and non-functional requirements for the development team.
UI/UX & Frontend Lead	Kaylee Carter	• Designs the complete user interface (UI) and user experience (UX), focusing on the "simplified user flow" requested by the client to improve upon the existing SRMS website. • Creates the wireframes, flow charts, and interactive prototypes for client approval before development begins. • Leads the frontend development using a cross-platform framework (e.g., Flutter) to build a responsive and accessible application for both Android and iOS.
Backend & API Integration Specialist	Khaleel Khan	• Takes primary responsibility for the critical task of integrating with the university's legacy PHP-based SRMS. • Designs, develops, and documents the secure API that will serve as the middleware between the old system and the new mobile app. • Manages the database interactions to ensure real-time data synchronization (Risk R3) and implements the server-

		side logic, including the conditional payment system for foreign students.
Full-Stack Developer	Tulsidai Singh	• Supports both frontend and backend development to accelerate progress on key modules. • Assists the Backend Specialist with API development and database management. • Collaborates with the Frontend Lead on implementing UI components and ensuring the application runs smoothly on a variety of devices, including low-spec models.
QA & Deployment Lead	Vimla Persaud	• Develops and executes the comprehensive multi-stage testing plan (Unit, Integration, Alpha, Beta). • Focuses testing efforts on the highest-risk areas: data synchronization, system load during peak times, and security vulnerabilities (GDPR compliance). • Manages the beta testing program with student volunteers. • Handles all technical documentation, prepares the application for deployment, and oversees the submission process to the Google Play Store and Apple App Store.

2.2 Communication and Reporting

- Weekly team meetings to review progress, identify blockers, and adjust schedules.
- Bi-weekly client or stakeholder reviews for updates and approvals.
- Daily developer stand-ups to ensure continuous integration and task alignment.
- Project updates and documentation are stored in a shared cloud repository (e.g., Google Drive, GitHub).

2.3 Decision-Making and Accountability

All strategic and design decisions are finalized by the Project Manager in consultation with the Systems Analyst and development leads. Each team member is accountable for delivering assigned tasks within agreed timelines, while the QA Engineer ensures all deliverables meet established quality standards before deployment.

3.0 Risk Analysis

The table below presents 6 primary risks identified for the University of Guyana's Student Records Management System (SRMS) Mobile Application project. These risks have been assessed based on our client meetings with the Registry Department and includes their likelihood and impact ratings, descriptions and mitigation strategies.

Risk ID	Risk Title	Likelihood (1-5)	Impact (1-5)	Description	Mitigation
R1	System Overload During Peak Usage	5	5	The Registry stated that the current SRMS website crashed in September 2025 under a load of 15,000 students during the registration period. With more students due to free tuition, and since mobile apps are used more often than websites, the risk of another system overload is very high. If the system goes down during the 1-2 week registration period, students can't sign up for classes, which would be a huge inconvenience.	• Before launch, we will load and stress test over 20,000 users accessing the app at once to see how the system handles it. • We will work with UG's IT team to make sure the servers can handle the extra traffic, possibly by adding more server resources automatically during peak times. • We will use the Registry's idea of a phased rollout, faculty by faculty to monitor performance and fix issues before full release.
R2	Legacy PHP System Integration	4	5	The current SRMS is built on a PHP system with no existing API layer or middleware infrastructure. This means it was not designed to connect with	• Before full development, we'll dedicate a 2-3 week phase to investigate the old system. We'll build a small proof-

				other applications. We can't just insert our mobile app, we will have to build a new connection layer from scratch, which could be very time-consuming and lead to bugs if the old and new systems don't match up properly. The Registry has confirmed that all current functionality is tightly coupled within the PHP codebase, making it hard to separate functions like registration.	of-concept API for just one feature (like viewing grades) to prove our connection method works. • We will document how our API works so everyone on the team is clear.
R3	Data Synchronization Failures Between Web and Mobile Platforms	4	5	As emphasized by the Registry contact (Tyreke Wilson), a seamless transition between the existing web application and the new mobile app in real time is important. If a student registers for a course on the app, it must show up on the website instantly and vice versa. Synchronization failures could result in students seeing outdated grades, incorrect course registrations or missing application statuses. Given that both platforms will operate at the same time, any sync lag could make users lose trust.	• We will design the system so that both the app and the website get their information from the exact same database via the API. This avoids having two separate data sources that could go out of sync. • We will write automated tests that run after every update to constantly check that data is consistent between platforms. • We'll include logic to log and reverse any transaction that fails to sync properly.

R4	Data Security and Privacy Breach	4	5	The mobile application will handle sensitive student information such as ID documents, academic records, financial details (for foreign students) and personal contact information. If there is a security breach, this data could be accessed by unauthorized parties, which may lead to identity theft or academic fraud. The Registry emphasized that the system will also serve international students so the application must comply with GDPR requirements as well as Guyana's Data Protection Act. Non-compliance could result in legal penalties, loss of student trust especially among foreign students and reputational damage to the University of Guyana.	• Privacy by Design: Carry out a Data Protection Impact Assessment (DPIA) early in the design phase. • Encryption: All data must be encrypted both in transit (TLS 1.3 minimum) and at rest (AES-256). • Authentication: Use two-factor authentication (2FA) for sensitive tasks like viewing grades or updating registration.
R5	Payment System Complexity for Foreign Students	4	4	Unlike most Caribbean universities, UG has a unique payment structure: tuition is free for Guyanese citizens but foreign students (including CARICOM nationals) must pay. The mobile application must correctly identify student	• We will build the payment system as a separate module that only loads if the user's profile is marked as "foreign." • We will implement the payment workflow as a separate phase after

				nationality/residency and only show the payment option to those who need it. This requires integration with payment gateways, invoice generation systems, and potentially document verification for residency proof. Getting this logic wrong could mean foreign students can't pay, or local students are incorrectly asked for money.	core registration functionality is stable. • We will create test accounts for both "Guyanese student" and "Foreign student" personas and test the payment workflow thoroughly to make sure the correct logic is applied every time. • Require formal approval of the payment workflow from both the Registry and the Bursary.
R6	Mobile Device Performance Issues	3	4	As specified by the Registry, the application must function on "the lowest minimum accepted device currently" because UG serves students from all regions of Guyana, many with 5-6 year old devices with limited RAM (2GB or less) and storage. The cross-platform application must perform well on low-spec devices while maintaining functionality for modern devices. Poor performance on older devices would defeat the accessibility	• We'll use a framework like React Native and keep the app size under 50MB. We'll aim for a load time of under 3 seconds on a slow 3G network. • We will recruit beta testers for older, low-spec phones to get real-world feedback on performance. • We will consider adding a "lite mode" that turns off non-essential features to

				goal of the mobile application.	improve speed on very old phones.
R7	Insufficient Testing Coverage Leading to Production Defects	4	4	The Registry emphasized the need for alpha and beta testing before rollout. Given the application handles sensitive academic data (grades, registration, personal information) and operates during time-sensitive periods (registration deadlines), production bugs could have severe consequences: students unable to register, incorrect grade displays, or data corruption. The tight project timeline creates pressure to abbreviate testing phases. Additionally, testing must cover the unique workflows specific to UG (foreign payment conditionals, document uploads for applications, synchronization with legacy PHP system).	We will follow a multi-stage testing plan: • Unit Testing: Developers will test their own code to ensure at least 80% coverage. • Integration Testing: We'll have automated tests that check if the web and mobile sync is working after every code change. • Alpha Testing: Our own team will test the app for 2 weeks using realistic test data. • Beta Testing: We'll recruit 100-150 students with different phone types and from different faculties to use the app for 3-4 weeks and report bugs. • Load Testing - Simulate peak registration loads before launch. Use the staggered rollout strategy to treat initial phases as extended beta testing with close monitoring. Establish

					a bug triage process with severity classifications and fix timelines.
R8	Low User Adoption Due to Poor User Experience	3	4	Students currently rely on the SRMS web application despite reported dissatisfaction with its user experience (confirmed by Registry). If the mobile app doesn't offer a significantly improved, intuitive experience, students may simply continue using the familiar web application, which makes the mobile development effort unsuccessful. The Registry noted that simplified user flows are essential; "doing something simple shouldn't require all the steps that it does on the website."	• We will start by interviewing 20-30 students to find out exactly what they dislike about the current website. • We will design the app for mobile first, focusing on making key tasks like registration and checking grades possible in just a few taps for thumb-friendly navigation. • We will implement the Registry's suggestion for a home screen dashboard that shows key information at a glance. • We'll ask students to test our designs and prototypes during development, not just after. • Consider gamification or notification features (grade alerts, registration reminders) that

					leverage mobile capabilities the web lacks. • Use the beta phase to gather System Usability Scale (SUS) scores and iterate based on feedback before full launch.
--	--	--	--	--	---

4.0 Hardware and Software Resource Requirements

4.1 Hardware Requirements

Category	Hardware Component	Specifications / Requirements	Purpose/ Justification
Developer Stations	Laptops/ Desktops	Intel Core i5 / Ryzen 5 or higher 8-16 GB RAM 512 GB SSD Windows 11 or Ubuntu 24.04 OS	Will be used by the development team for coding, testing, database design and for documentation
Mobile Testing devices	Android and IOS smartphones	Android 8.0 + IOS 14 +	Used to test app performance, compatibility and responsiveness on major OS
Server Hardware/Integration (Middleware Host)	Dedicated Application Server / Virtual Machine	8 vCPU (expandable) 16–32 GB RAM 1–2 TB SSD Ubuntu Server 24.04 LTS	Will host the new unified API that integrate both srms and mobile app. This will allow the existing SRMS web system and the mobile app to connect to the same API for synchronization, ensuring consistent and secure data exchange between platforms.
Networking Equipment	Routers, Modem, Switches	250 - 500 mbps internet bandwidth Secured and stable wifi connectivity	Provides stable internet access and communication between work stations and servers

Backup & Storage Devices	SDD or Cloud storage	1TB Minimum	Used for frequent backups of project files, source code and documentation
-------------------------------------	----------------------	-------------	---

4.2 Software Requirements

Category	Software	Version/Type	Purpose/ Justification
Operating System	Windows 11/ Ubuntu 24.04	Latest Version	Development environment for Workstations and servers
Database Mangement system (DBMS)	Existing SRMS Database (MySQL / MariaDB)	Already in used	Manages Students, courses and registration records through secure APIs
Backend Development tools	PHP, Node.js	Open source	Allows for the synchroinzation of both the mobile and web application.
Mobile App Framework	Flutter/dart	Cross-platform	Allows single codebase for both Android and IOS
Frontend Libraries/UI kits	Material Design/ Bootstrap/ Cupertino (IOS)	Latest Version	Provides consistent and user friendly interface following UG's branding
Version Control System	Git/ Github	Cloud-based	Allows for team collaboration, tracking changes and source code management.

API Testing Tools	Postman/ Jmeter	Latest Version	Used to verify synchronization between the app and srms, also testing of API endpoints.
Virtual Testing Environment	Android Studio Emulator, IOS simulator	Virtual device simulation	Used to simulate multiple Android and IOS devices for the testing of the apps functionality, synchronization and performance without requiring physical devices.
Security and Compliance Tools	Open SSL, OAuth2, SSL/TLS	Industry-Standard	Ensure encryption of students data and secure user authentication.
Documentation Tools	MS, Lucidchart, Draw.io, Figma	Licenced/ Free	Used for Wireframes, UML diagrams, report and docuementation
Project Management Tools	Trello/ Jira	Web-based	Used for task tracking, scheduling anf milestone management using the plan driven (waterfall) approach.

5.0 Work Breakdown

Project Initiation & Planning

This phase focuses on defining the project scope, constraints, and the plan we are currently building.

- **Activities:**
 - Conduct initial meeting(s) with a representative from SSETA - TOS, Mr. Tyreke Wilson, to gather high-level goals.
 - Additionally, collect information about desired features from the representative.
 - Identify key stakeholders (e.g., Registry, Software Department, Students).
 - Identify target users (students as primary users).
 - Identify the main project constraints: Time, staffing, and the need to sync the app with the website.
 - Conduct an initial risk analysis.
 - Gather the software and hardware requirements for this system to be implemented.
- **Milestones:**
 - Project Initial Meeting Completed.
 - Project Plan to be Approved.
- **Deliverables:**
 - **Project Plan**

Requirements Gathering & Analysis

This phase outlines the app's functionality, with a main focus on the student's perspective.

- **Activities:**
 - Identify key system functions:
 - **Registration Module:** Registering for courses and viewing registered courses.

- **Application Module:** Applying for university, uploading documents, and checking application status.
- **Timetable Module:** Viewing class timetables.
- Include a payment feature, but design it to skip Guyanese residents.

Analyze and categorize features into modules:

- Module 1: Application & Admission
- Module 2: Course Registration
- Module 3: Timetable & Scheduling
- Module 4: Notifications & Alerts
- Module 5: Payment & Verification
- Design sketches (wireframes) for every screen in the app.
- Design the database and the API (the "bridge") to connect to the current SRMS system.
- Define each module's expected inputs, processes, and outputs.
- Define integration requirements (via **API or middleware**) to sync with the existing custom PHP system.
- Define Non-Functional Requirements, including:
 - **Performance:** Must be lightweight, fast, and optimized for size to run on older devices.
 - **Security:** The application should comply with the General Data Protection Regulation (**GDPR**).
 - **Usability:** Must have a "clean, simple, straightforward" and "simplified user flow" to improve on the current web app's User Interface.
 - **Platform:** Must be cross-platform, can be used on both Android and iOS.
- **Milestones:**
 - All features are agreed upon.
 - Draft requirements documented.
- **Deliverables:**
 - Software Requirements Specification document.

- Functional and non-functional requirements approved by Mr. Tyreke Wilson

Design (UX/UI & Technical)

This phase maps out the user experience and technical architecture based on the defined requirements.

- **Activities:**

- Design a "simplified user flow" to reduce the number of steps for common tasks.
- Create **Wireframes** for all app screens and modules.
- Create **Flow Charts** to visualize the application and registration processes.
- Design the technical architecture, including API specifications.
- Create **Database Models** (structure).
- Design the security framework, detailing encryption and data handling.

- **Milestones:**

- User Flow and Wireframes Approved.
- Technical Architecture & Database Models Approved.

- **Deliverables:**

- **Wireframes** (for each module).
- **Flow Charts**.
- **Database Structure Models**.

Development & Integration (Modular)

This phase involves the theoretical build of the application, focusing on the high-priority modules first.

- **Activities:**

- Set up development environments for cross-platform (Android/iOS) deployment.
- Develop and test APIs to integrate with the existing SRMS.
- Develop **Module 1: Application**.
- Develop **Module 2: Registration**.

- Develop **Module 3: Timetable**.
 - Build the push notification system for grades and updates.
- **Milestones:**
 - Module 1 (Application) Demo Ready.
 - Module 2 (Registration) Demo Ready.
 - Module 3 (Timetable) Demo Ready.
- **Deliverables:**
 - Functional API.
 - Source Code.
 - Demos for each developed module.

Quality Assurance & Testing

This phase is critical for mitigating the high-stakes risks of downtime and data-sync failures.

- **Activities:**
 - Conduct internal testing (Alpha testing) by the development team.
 - Manage the Beta Testing process with a small group of real students to gather feedback on performance and UX.
 - Conduct Load Testing to simulate high-demand periods (like registration) and ensure servers can be scaled.
 - Perform security testing to ensure data protection and GDPR compliance.
 - Perform Quality Analysis tests on low-spec physical devices to ensure performance and usability across a range of student phones.
- **Milestones:**
 - Alpha Testing Completed.
 - Beta Testing Completed.
 - Load Testing Certified.
- **Deliverables:**

- Test Plans and Test Cases.
- User Testing Feedback Report.

Deployment

This phase details the plan for launching the app to the entire student body, with a focus on stability.

- **Activities:**

- Prepare and submit applications to the Apple App Store and Google Play Store.
- Develop user support documentation.
- Plan and execute an Incremental Rollout (e.g., releasing to one faculty, like Computer Science, first) to monitor server load.
- Actively monitor server performance and scale infrastructure as needed.
- Manage the "seamless transition" to ensure data is synced between the web and mobile apps.

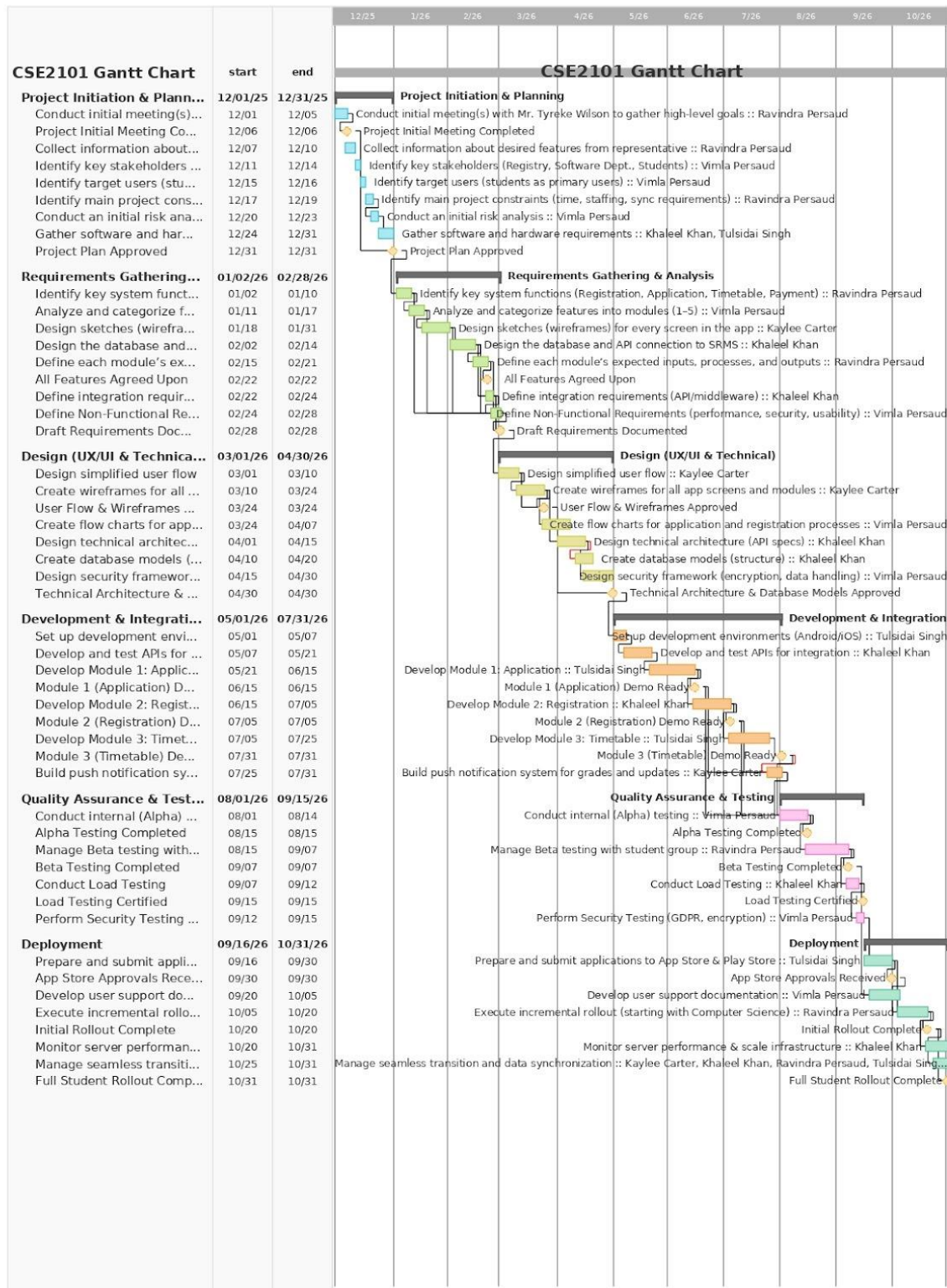
- **Milestones:**

- App Store Approvals Received.
- Initial Rollout Complete.
- Full Student Rollout Complete.

- **Deliverables:**

- Production-Ready Mobile Application (on app stores).
- Rollout Schedule.
- User Support & Help Documentation.

6.0 Project Schedule



[Here is the link to our Gantt Diagram created using TeamGantt](#)

Phase 1: Project Initiation & Planning

Task	Start	End	Duration	Dependency	Responsible
Conduct initial meeting(s) with Mr. Tyreke Wilson to gather high-level goals	Dec 1	Dec 5	1 week	—	Ravindra
Collect information about desired features from representative	Dec 5	Dec 10	1 week	After initial meeting	Ravindra
Identify key stakeholders (Registry, Software Dept., Students)	Dec 10	Dec 14	5 days	—	Vimla
Identify target users (students as primary users)	Dec 14	Dec 16	3 days	—	Vimla
Identify main project constraints (time, staffing, sync requirements)	Dec 16	Dec 19	4 days	—	Ravindra
Conduct an initial risk analysis	Dec 19	Dec 23	5 days	Constraints identified	Vimla
Gather software and hardware requirements	Dec 23	Dec 31	1 week	Risk analysis	Khaleel & Tulsidai

Phase 2: Requirements Gathering & Analysis

Task	Start	End	Duration	Dependency	Responsible
Conduct initial meeting(s) with Mr. Tyreke Wilson to gather high-level goals	Dec 1	Dec 5	1 week	—	Ravindra
Collect information about desired features from representative	Dec 5	Dec 10	1 week	After initial meeting	Ravindra
Identify key stakeholders (Registry, Software Dept., Students)	Dec 10	Dec 14	5 days	—	Vimla
Identify target users (students as primary users)	Dec 14	Dec 16	3 days	—	Vimla
Identify main project constraints (time, staffing, sync requirements)	Dec 16	Dec 19	4 days	—	Ravindra
Conduct an initial risk analysis	Dec 19	Dec 23	5 days	Constraints identified	Vimla
Gather software and hardware requirements	Dec 23	Dec 31	1 week	Risk analysis	Khaleel & Tulsidai

Phase 3: Design (UX/UI & Technical)

Task	Start	End	Duration	Dependency	Responsible
Conduct initial meeting(s) with Mr. Tyreke Wilson to gather high-level goals	Dec 1	Dec 5	1 week	—	Ravindra

Collect information about desired features from representative	Dec 5	Dec 10	1 week	After initial meeting	Ravindra
Identify key stakeholders (Registry, Software Dept., Students)	Dec 10	Dec 14	5 days	—	Vimla
Identify target users (students as primary users)	Dec 14	Dec 16	3 days	—	Vimla
Identify main project constraints (time, staffing, sync requirements)	Dec 16	Dec 19	4 days	—	Ravindra
Conduct an initial risk analysis	Dec 19	Dec 23	5 days	Constraints identified	Vimla
Gather software and hardware requirements	Dec 23	Dec 31	1 week	Risk analysis	Khaleel & Tulsidai

Phase 4: Development & Integration

Task	Start	End	Duration	Dependency	Responsible
Conduct initial meeting(s) with Mr. Tyreke Wilson to gather high-level goals	Dec 1	Dec 5	1 week	—	Ravindra
Collect information about desired features from representative	Dec 5	Dec 10	1 week	After initial meeting	Ravindra
Identify key stakeholders (Registry, Software Dept., Students)	Dec 10	Dec 14	5 days	—	Vimla

Identify target users (students as primary users)	Dec 14	Dec 16	3 days	—	Vimla
Identify main project constraints (time, staffing, sync requirements)	Dec 16	Dec 19	4 days	—	Ravindra
Conduct an initial risk analysis	Dec 19	Dec 23	5 days	Constraints identified	Vimla
Gather software and hardware requirements	Dec 23	Dec 31	1 week	Risk analysis	Khaleel & Tulsidai

Phase 5: Quality Assurance & Testing

Task	Start	End	Duration	Dependency	Responsible
Conduct initial meeting(s) with Mr. Tyreke Wilson to gather high-level goals	Dec 1	Dec 5	1 week	—	Ravindra
Collect information about desired features from representative	Dec 5	Dec 10	1 week	After initial meeting	Ravindra
Identify key stakeholders (Registry, Software Dept., Students)	Dec 10	Dec 14	5 days	—	Vimla
Identify target users (students as primary users)	Dec 14	Dec 16	3 days	—	Vimla
Identify main project constraints (time, staffing, sync requirements)	Dec 16	Dec 19	4 days	—	Ravindra

Conduct an initial risk analysis	Dec 19	Dec 23	5 days	Constraints identified	Vimla
Gather software and hardware requirements	Dec 23	Dec 31	1 week	Risk analysis	Khaleel & Tulsidai

Phase 6: Deployment

Task	Start	End	Duration	Dependency	Responsible
Conduct initial meeting(s) with Mr. Tyreke Wilson to gather high-level goals	Dec 1	Dec 5	1 week	—	Ravindra
Collect information about desired features from representative	Dec 5	Dec 10	1 week	After initial meeting	Ravindra
Identify key stakeholders (Registry, Software Dept., Students)	Dec 10	Dec 14	5 days	—	Vimla
Identify target users (students as primary users)	Dec 14	Dec 16	3 days	—	Vimla
Identify main project constraints (time, staffing, sync requirements)	Dec 16	Dec 19	4 days	—	Ravindra
Conduct an initial risk analysis	Dec 19	Dec 23	5 days	Constraints identified	Vimla
Gather software and hardware requirements	Dec 23	Dec 31	1 week	Risk analysis	Khaleel & Tulsidai

Collaborative Summary

- Introduction & Project Schedule: Vimla Persaud
- Project Organization: Khaleel Khan
- Risk Analysis: Tulsidai Singh
- Hardware and Software Requirements: Ravindra Persaud
- Work Breakdown: Kaylee Carter

Name	Signature
Kaylee Carter	
Khaleel Khan	
Ravindra Persaud	<i>R. Persaud</i>
Vimla Persaud	<i>Vimla Persaud</i>
Tulsidai Singh	<i>T. Singh</i>

Collaboration Strategies and General Experiences:

Our group used a shared Google Doc for real-time collaborative writing and a WhatsApp group for daily communication and coordination. This strategy worked well, as the shared document allowed everyone to see progress and contribute easily. Regular check-in meetings were held via Google Meet to ensure everyone was aligned.

Challenges and Resolutions:

Our main challenge was ensuring a consistent writing style and tone across the document, since each section was written by a different person. We handled this by designating one member as the final compiler, responsible for reviewing the entire document and making minor edits to ensure it read as a single, cohesive report.